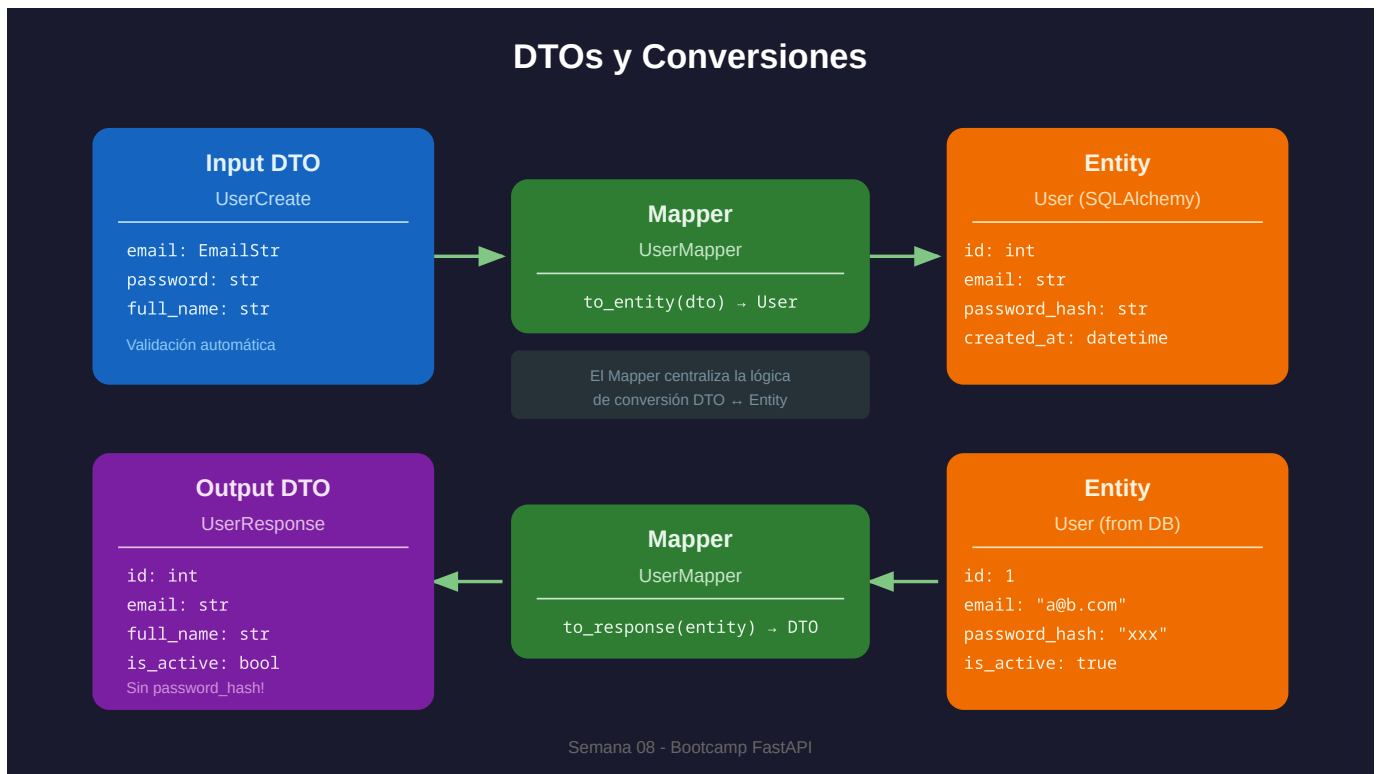


DTOs y Mappers



Objetivos

- Entender qué son los DTOs y por qué usarlos
- Implementar DTOs específicos por operación
- Crear Mappers para convertir entre capas
- Separar modelos de DB de schemas de API

¿Qué es un DTO?

DTO (Data Transfer Object) es un objeto que transporta datos entre capas o procesos. Su único propósito es contener datos, sin lógica de negocio.

¿Por qué usar DTOs?

```
# ❌ SIN DTOs - Exponiendo modelo de DB directamente
@router.get("/users/{id}")
def get_user(id: int, db: Session = Depends(get_db)):
    user = db.get(User, id)
    return user # Expone password_hash, campos internos, etc.

# ✅ CON DTOs - Control sobre qué se expone
@router.get("/users/{id}", response_model=UserResponse)
def get_user(id: int, service: UserService = Depends(get_service)):
    return service.get_by_id(id) # Retorna DTO sin campos sensibles
```

Tipos de DTOs

1. Request DTOs (Input)

```
# schemas/product.py
from pydantic import BaseModel, Field

class ProductCreate(BaseModel):
    """DTO para CREAR producto - todos los campos requeridos"""
    name: str = Field(..., min_length=1, max_length=200)
    sku: str = Field(..., pattern=r"^[A-Z]{3}-\d{4}$")
    price: float = Field(..., gt=0)
    stock: int = Field(default=0, ge=0)
    category_id: int

class ProductUpdate(BaseModel):
    """DTO para ACTUALIZAR producto - campos opcionales"""
    name: str | None = Field(None, min_length=1, max_length=200)
    price: float | None = Field(None, gt=0)
    stock: int | None = Field(None, ge=0)
    category_id: int | None = None
    # SKU no se puede cambiar (regla de negocio)
```

2. Response DTOs (Output)

```
from datetime import datetime
from pydantic import ConfigDict

class ProductResponse(BaseModel):
    """DTO de respuesta - lo que ve el cliente"""
    model_config = ConfigDict(from_attributes=True)

    id: int
    name: str
    sku: str
    price: float
    stock: int
    category_id: int
    created_at: datetime
    # NO incluye: updated_at interno, soft_delete flags, etc.

class ProductDetail(ProductResponse):
    """DTO con más detalle - incluye relaciones"""
    category: "CategoryResponse"

class ProductList(BaseModel):
    """DTO para listados con paginación"""
    items: list[ProductResponse]
    total: int
    page: int
```

```
page_size: int
pages: int
```

3. Internal DTOs (Entre capas)

```
class ProductFilter(BaseModel):
    """DTO para filtros internos"""
    category_id: int | None = None
    min_price: float | None = None
    max_price: float | None = None
    in_stock: bool | None = None
    search: str | None = None
```

Mappers

Los **Mappers** convierten entre diferentes representaciones de datos.

Mapper Simple (Funciones)

```
# mappers/product.py
from models.product import Product
from schemas.product import ProductCreate, ProductResponse

def to_entity(dto: ProductCreate) -> Product:
    """Convierte DTO -> Entity"""
    return Product(
        name=dto.name,
        sku=dto.sku,
        price=dto.price,
        stock=dto.stock,
        category_id=dto.category_id
    )

def to_response(entity: Product) -> ProductResponse:
    """Convierte Entity -> Response DTO"""
    return ProductResponse(
        id=entity.id,
        name=entity.name,
        sku=entity.sku,
        price=entity.price,
        stock=entity.stock,
        category_id=entity.category_id,
        created_at=entity.created_at
    )

def to_response_list(entities: list[Product]) -> list[ProductResponse]:
    """Convierte lista de entities"""
    return [to_response(e) for e in entities]
```

Mapper como Clase

```
class ProductMapper:
    """Centraliza conversiones de Product"""

    @staticmethod
    def to_entity(dto: ProductCreate) -> Product:
        return Product(
            name=dto.name,
            sku=dto.sku,
            price=dto.price,
            stock=dto.stock,
            category_id=dto.category_id
        )

    @staticmethod
    def to_response(entity: Product) -> ProductResponse:
        return ProductResponse.model_validate(entity)

    @staticmethod
    def update_entity(entity: Product, dto: ProductUpdate) -> Product:
        """Aplica cambios del DTO a la entidad"""
        update_data = dto.model_dump(exclude_unset=True)
        for field, value in update_data.items():
            setattr(entity, field, value)
        return entity
```

Uso en Service

```
# services/product.py
from schemas.product import ProductCreate, ProductUpdate, ProductResponse
from mappers.product import ProductMapper

class ProductService:
    def __init__(self, repo: ProductRepository):
        self.repo = repo

    def create(self, data: ProductCreate) -> ProductResponse:
        # Validaciones de negocio
        if self.repo.exists_by_sku(data.sku):
            raise ProductAlreadyExistsError(f"SKU '{data.sku}' exists")

        # DTO -> Entity
        entity = ProductMapper.to_entity(data)

        # Persistir
        saved = self.repo.add(entity)

        # Entity -> Response DTO
        return ProductMapper.to_response(saved)

    def update(self, product_id: int, data: ProductUpdate) -> ProductResponse:
```

```

entity = self.repo.get_by_id(product_id)
if not entity:
    raise ProductNotFoundError(f"Product {product_id} not found")

# Aplicar cambios
updated = ProductMapper.update_entity(entity, data)
saved = self.repo.update(updated)

return ProductMapper.to_response(saved)

```

Comparación: Con y Sin DTOs

Sin DTOs (Modelo expuesto)

```

# models/user.py
class User(Base):
    id: Mapped[int]
    email: Mapped[str]
    password_hash: Mapped[str] # ⚠ Sensible
    is_admin: Mapped[bool] # ⚠ Interno
    login_attempts: Mapped[int] # ⚠ Interno
    created_at: Mapped[datetime]

# ❌ Expone todo al cliente
@router.get("/users/{id}")
def get_user(id: int):
    return db.get(User, id) # Incluye password_hash!

```

Con DTOs (Control total)

```

# schemas/user.py
class UserResponse(BaseModel):
    """Solo campos públicos"""
    model_config = ConfigDict(from_attributes=True)

    id: int
    email: str
    created_at: datetime
    # NO: password_hash, is_admin, login_attempts

# ✅ Solo datos públicos
@router.get("/users/{id}", response_model=UserResponse)
def get_user(id: int, service: UserService = Depends()):
    return service.get_by_id(id)

```

Pydantic model_validate

Pydantic v2 facilita la conversión con `model_validate` :

```

from pydantic import BaseModel, ConfigDict

```

```
class ProductResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    name: str
    price: float

# Convierte automáticamente SQLAlchemy → Pydantic
product_entity = repo.get_by_id(1)
response = ProductResponse.model_validate(product_entity)
```

✓ Mejores Prácticas

1. **Un DTO por operación:** Create , Update , Response
2. **Response DTO siempre:** Nunca exponer modelos de DB
3. **Validaciones en DTOs:** Usar Field() de Pydantic
4. **Mappers centralizados:** Fácil de mantener y testear
5. **from_attributes=True:** Facilita conversión ORM → DTO

✓ Checklist

- ☐ Entiendo la diferencia entre DTO y Entity
- ☐ Sé crear DTOs para Create, Update y Response
- ☐ Puedo implementar Mappers
- ☐ Entiendo cuándo usar `model_validate`